SYSTEM DESIGN INTERVIEW

Design a URL Shortener

bit.ly-style distributed service for shortening and redirecting URLs

~ 40 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Statement & Clarifying Questions
02 · Functional & Non-Functional Requirements
03 · Capacity Estimation
04 · High-Level Architecture
05 · Key Generation Strategy
06 · Database Design & Schema
07 · Write Path Deep Dive
08 · Read Path & Redirect
09 · Analytics Pipeline
10 · URL Expiry & Cleanup
11 · Security & Abuse Prevention
12 · Key Generation Service (KGS)
13 · Scalability & Distribution
14 · Failure Modes & Recovery
15 · Design Trade-offs
16 · Interview Playbook

 © Study Guide — For personal use only

01 Problem Statement

Requirements and scope

Design a URL shortening service like **bit.ly** or **TinyURL**. The service must convert long URLs into short, memorable links; handle user redirects efficiently; track analytics; and scale to billions of URLs.

Clarifying Questions

Question	Answer
URLs/day?	100 million read-heavy (100:1 read:write ratio)
Expiry?	5 years; lazy expiry on read
Custom aliases?	Yes, optional per-user feature
Scale?	Global; handle single region first
Private URLs?	Yes, with auth + access control
Vanity URLs?	Yes, if available and reserved by user
Analytics required?	Yes: click counts, geographic, referrer

© Study Guide — For personal use only

02 Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Shorten	Accept long URL → return short URL (POST /shorten)
Redirect	GET /:key → 301 redirect to original URL
Analytics	Track clicks, referrer, geo, user agent per URL
Expiry	URLs expire after configurable TTL (default 5 years)
Custom Key	Users can specify custom alias if available
Private URLs	Owner-only access with auth token

Non-Functional Requirements

Requirement	Target
Availability	99.9% uptime; graceful degradation
Latency	<100ms for redirect; <200ms for shorten
Consistency	Eventual consistency acceptable; dedup via keys
Storage	~91 TB over 5 years
Throughput	1.2K writes/sec; 115K reads/sec
Scalability	Geo-distributed; 62^7 ≈ 3.5 trillion keys

© Study Guide — For personal use only

03 Capacity Estimation

Math for scale

Traffic Estimation

- **URLs created/day:** 100 million
- **Read:Write ratio:** 100:1 (typical for shorteners)
- **Write throughput:** 100M / 86,400 sec = 1,157 writes/sec ≈ 1.2K/sec
- **Read throughput:** 1.2K × 100 = 115,700 reads/sec ≈ 120K/sec

Storage Estimation

- **Per URL record:** short_key (7B) + original_url (avg 100B) + metadata (100B) = ~500 bytes
- **Daily storage:** 100M × 500B = 50 GB/day
- **Retention:** 5 years = 1,825 days
- **Total:** 50 GB × 1,825 = 91.25 TB

Cache Estimation

- **Hot URLs:** 20% of reads hit 80% of URLs (Pareto principle)
- **Hot set size:** (115K reads/sec) × 0.8 × (24 hours) = 7.9B requests to 1.6B unique URLs
- **Cache size needed:** 1.6B URLs × 500B = 800 GB (or deduplicate to top 23M: 23M × 500B = 11.5 GB)

N KEY NUMBERS

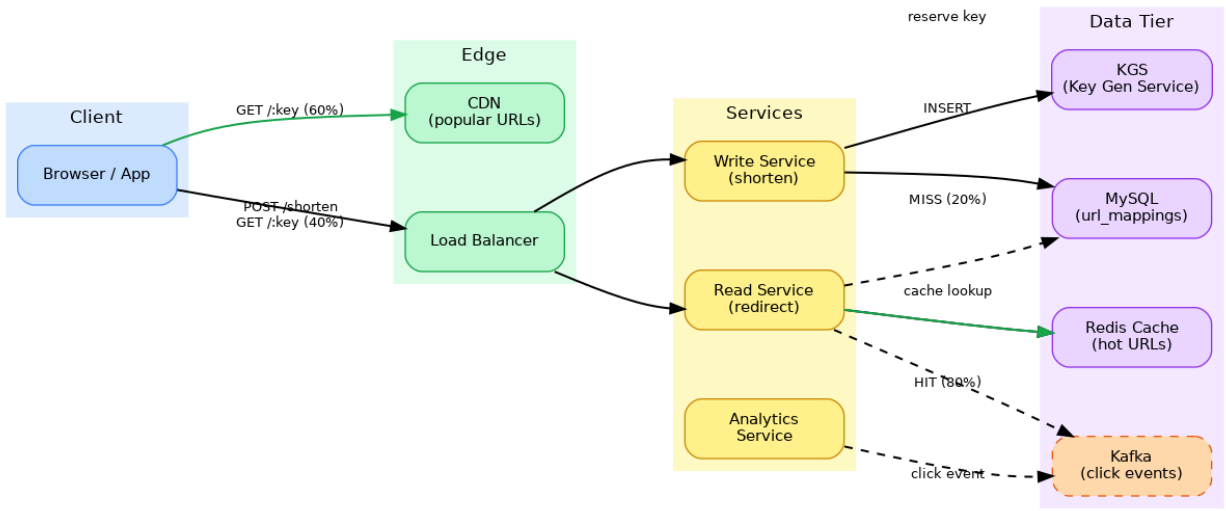
Write throughput: 1.2K/sec
Read throughput: 120K/sec
Total storage: 91 TB
Cache (20% hot): 11.5 GB (Redis cluster)

© Study Guide — For personal use only

04 High-Level Architecture

System overview

The system is divided into **Edge (CDN + Load Balancer)**, **Service Layer (Write, Read, Analytics)**, and **Data Tier (KGS, MySQL, Redis, Kafka)**. Reads are cached and served from CDN for popular URLs; writes reserve keys from a pre-generated pool.



Architecture Highlights

- **CDN Edge:** Caches 60% of requests (popular URLs); serves 301 redirects
- **Load Balancer:** Routes write requests to Write Service, read requests to Read Service
- **Write Service:** Validates input, reserves short key from KGS, persists to MySQL, warms cache
- **Read Service:** Checks Redis cache (80% hit rate), falls back to MySQL, logs analytics to Kafka
- **KGS:** Maintains pre-generated key pool; assigns keys atomically to avoid collisions
- **MySQL:** Persistent storage; sharded by short_key hash for scalability
- **Redis:** In-memory cache for hot URLs; replicated cluster for HA
- **Kafka:** Event stream for click logging; async ingestion to Analytics

© Study Guide — For personal use only

05 Key Generation Strategy

MD5 vs Base62 vs KGS

Overview

- **Goal:** Convert long URL → short, unique, memorable string
- **Candidates:** MD5 hash, Base62 encoding, Pre-generated Key Service (KGS)

Comparison

Approach	Pros	Cons
MD5 Hash	Deterministic; no DB lookup	Not memorable; collision possible
Base62 from Counter	Compact; sequential	Predictable; need distributed counter
KGS Pre-generated	Atomic; no collisions; load-balanced	Requires separate service; pre-warming

Why KGS Wins

- **Atomicity:** Keys reserved from a pre-generated pool, no race conditions
- **No Collisions:** Each key issued exactly once; dedup via database unique constraint
- **Load Balancing:** Standby KGS instances for HA; keys pre-loaded on startup
- **Predictability:** Latency unaffected by hash computation or counter coordination

Base62 Encoding (Fallback)

```
import string
charset = string.digits + string.ascii_lowercase + string.ascii_uppercase
def to_base62(num):
if num == 0: return charset[0]
s = []
while num > 0:
s.append(charset[num % 62])
num //= 62
return ''.join(reversed(s))
# 62^7 = 3.5 trillion unique keys
keys = 62 ** 7 # 3,521,614,606,208
```

Base62 encoding for fallback counter-based approach

Key Pool Size Math

© Study Guide — For personal use only

- **Alphabet:** 0–9, a–z, A–Z = 62 characters
- **Key length:** 7 characters (bit.ly standard)
- **Total keys:** 62^7 = 3,521,614,606,208 (3.5 trillion)
- **Years to exhaust:** 3.5T / (100M/day × 365) ≈ 95,910 years

© Study Guide — For personal use only

06 Database Design & Schema

SQL schema and indexing

Primary Table: url_mappings

```
CREATE TABLE url_mappings (
short_key VARCHAR(7) PRIMARY KEY,
original_url TEXT NOT NULL,
user_id BIGINT,
created_at TIMESTAMP DEFAULT NOW(),
expires_at TIMESTAMP,
is_custom BOOLEAN DEFAULT FALSE,
click_count INT DEFAULT 0,
last_accessed TIMESTAMP,
UNIQUE KEY uk_user_custom (user_id, short_key) WHERE is_custom,
INDEX idx_user_id (user_id),
INDEX idx_expires_at (expires_at)
);
```

Primary table for storing URL mappings

Indexing Strategy

- **PRIMARY KEY (short_key):** Fast lookup on redirect; sharded
- **INDEX (user_id):** For user's URL list queries
- **INDEX (expires_at):** For background cleanup job
- **UNIQUE (user_id, is_custom):** Prevent duplicate custom URLs per user

Sharding Strategy

- **Shard key:** hash(short_key) % num_shards (e.g., 256 shards)
- **Reasoning:** Distributes writes evenly; enables parallel reads
- **Resharding:** Use consistent hashing or shard migration tool (e.g., Vitess)
- **Shard count:** Start with 256; scale to 1024+ as needed

SQL vs NoSQL Decision

Aspect	SQL (MySQL)	NoSQL (DynamoDB/Cassandra)
Schema	Fixed; good for structured data	Flexible; harder to query
Transactions	Full ACID support	Limited (eventual consistency)

© Study Guide — For personal use only

Aspect	SQL (MySQL)	NoSQL (DynamoDB/Cassandra)
Joins	Yes; useful for analytics	No; denormalize instead
Scalability	Sharding required; complex	Built-in replication; easier
Cost	Cheaper at scale; self-managed	Expensive; high write throughput

✓ RECOMMENDATION

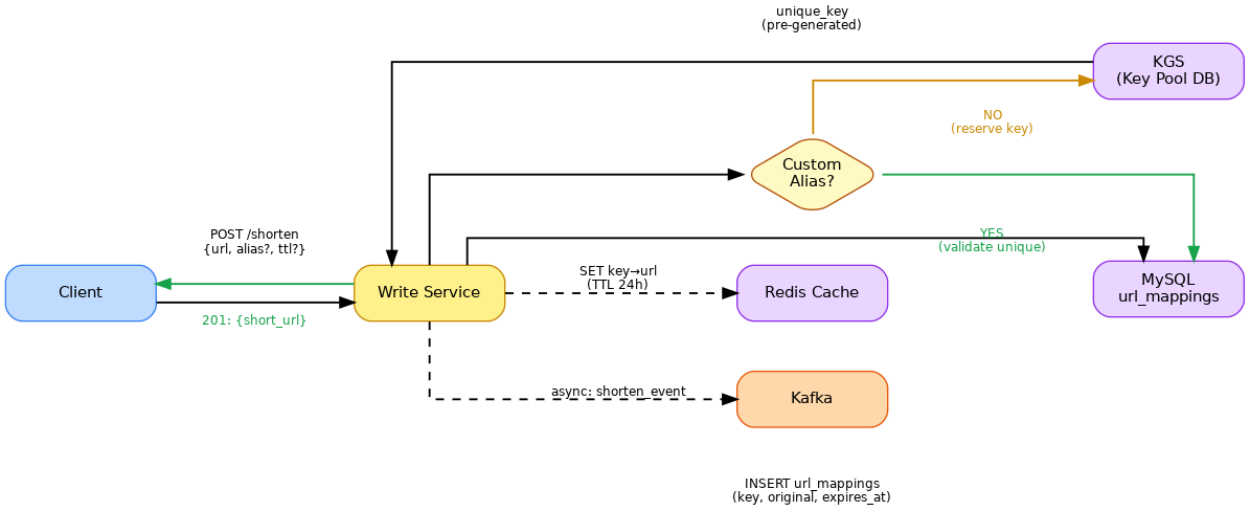
Use **MySQL with sharding** for this problem: Structured data, strong consistency needed, analytics queries beneficial. Shard by short_key hash to distribute load evenly.

© Study Guide — For personal use only

07 Write Path Deep Dive

Shortening a URL

Flow Overview



Step-by-Step

1. **Client submits:** POST /shorten { url, alias?, ttl? }
2. **Validate:** URL format, length, safe browsing check (Google Safe Browsing API)
3. **Custom alias?** If provided, check uniqueness in MySQL; reject if taken
4. **Reserve key:** If no custom alias, request key from KGS (atomic)
5. **Persist:** INSERT into url_mappings (short_key, original_url, user_id, expires_at)
6. **Warm cache:** SET Redis key with TTL (24 hours for hot URLs)
7. **Async event:** Publish to Kafka for analytics pipeline
8. **Return:** 201 { short_url, created_at, expires_at }

Optimizations

- **Batch KGS requests:** Reserve 100 keys per Write Service instance startup
- **Local cache:** In-memory queue of reserved keys; refill when depleted
- **Async persistence:** Write to MySQL; log to Kafka in parallel (event sourcing)
- **Dedup check:** Use UNIQUE constraint + database handles collisions

Error Handling

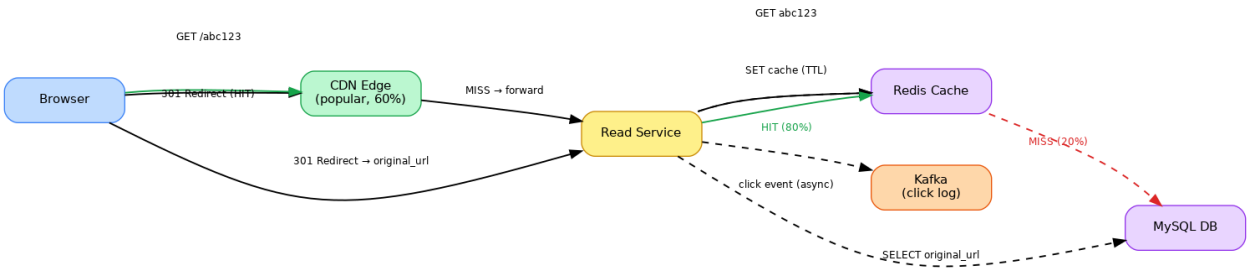
- **KGS exhausted:** Scale KGS horizontally; use larger key length (8 chars)
- **MySQL down:** Return 503; client can retry
- **Duplicate key:** Rare (1 in 3.5T); KGS prevents; MySQL constraint catches
- **Malicious URL:** Safe Browsing API returns UNSAFE; reject with 400

© Study Guide — For personal use only

08 Read Path & Redirect

Fast redirects

Flow Overview



Step-by-Step

1. **Browser requests:** GET /abc123
2. **CDN cache?** If HIT (60% popular URLs), serve 301 directly from edge
3. **Miss CDN:** Route to Read Service
4. **Redis lookup:** GET abc123 → 80% hit rate (hot URLs)
5. **Cache miss:** Query MySQL shard for original_url
6. **Warm cache:** SET Redis with TTL (24 hours)
7. **Return:** 301 Moved Permanently → original_url
8. **Async log:** Publish click event to Kafka (referrer, geo, user agent)

301 vs 302 Redirect

Type	Use Case	Pros	Cons
301	Permanent move	Browsers cache; reduces load	SEO: transfers link equity
302	Temporary move	SEO: preserves original URL	Higher traffic; no client cache

✓ RECOMMENDATION

Use **301** for bit.ly. Permanent redirects reduce server load by leveraging browser cache. SEO benefit (link equity transfer) is secondary to performance at this scale.

CDN Caching Strategy

- **Cache key:** GET /:short_key → origin (Read Service)
- **TTL:** 1 hour default; 24 hours for viral URLs
- **Invalidation:** On custom URL update or expiry; manual purge if needed

© Study Guide — For personal use only

- **Geographic:** Replicate across regions (US, EU, APAC)

Optimizations

- **Connection pooling:** Redis clusters with sentinel for HA
- **Batch logging:** Write click events in batches (1000 events) to reduce Kafka overhead
- **Prefetch:** Predict trending URLs; pre-warm Redis

© Study Guide — For personal use only

09 Analytics Pipeline

Click tracking and insights

Architecture

- **Click events:** Logged asynchronously by Read Service to Kafka
- **Kafka topic:** url_clicks (partition by short_key for ordering)
- **Stream processor:** Apache Flink or Kafka Streams
- **Aggregations:** Count, geographic, referrer, user agent; time windows (1min, 1hour, 1day)
- **Storage:** Cassandra (time-series DB) for analytics; Redis for dashboard

Event Schema

```
{
'short_key': 'abc123',
'user_id': 12345,
'timestamp': '2026-03-18T10:30:45Z',
'ip_address': '203.0.113.42',
'country': 'US',
'referrer': 'twitter.com',
'user_agent': 'Mozilla/5.0...',
'click_id': 'uuid()'
}
```

Click event schema (JSON)

Metrics

- **Total clicks:** Sum by short_key over time window
- **Unique visitors:** HyperLogLog for approx distinct count (saves memory)
- **Geographic heatmap:** Clicks by country; top 10 regions
- **Referrer breakdown:** Top referrers by click count
- **Trending:** Clicks in last 1 hour; identify viral URLs

N HYPERLOGLOG

For unique visitor counts, use **HyperLogLog** (probabilistic data structure). Uses ~1.5 KB per key, handles billions of unique values with ~2% error. Trade-off: accuracy for memory efficiency.

Dashboards

- **Per-URL dashboard:** Total clicks, unique visitors, top referrers, geographic

© Study Guide — For personal use only

- **Admin dashboard:** Aggregate metrics, system health, top 1000 URLs
- **Real-time:** Kafka Streams to Redis; dashboard queries Redis cache

© Study Guide — For personal use only

10 URL Expiry & Cleanup

Managing data retention

Expiry Mechanism

- **Default TTL:** 5 years from creation
- **Lazy expiry:** On read, check expires_at; if expired, return 404
- **No explicit delete:** Reduces write load; cleanup runs async
- **Custom TTL:** Users can specify shorter expiry (e.g., 7 days for links)

Background Cleanup Job

- **Frequency:** Daily at 2 AM UTC (off-peak)
- **Query:** SELECT short_key WHERE expires_at < NOW() LIMIT 100K
- **Cursor pagination:** Scan shards incrementally; prevent long-lock
- **Delete:** DELETE FROM url_mappings WHERE short_key IN (...)
- **Evict cache:** DEL from Redis for deleted keys
- **Log:** Track deleted keys for analytics retention

Cold Archive

- **Retention:** Keep deleted URLs in S3 Glacier (cold storage) for 7 years
- **Cost:** ~$0.004/GB/month (vs. $0.10/GB/month for MySQL)
- **Query:** Archive by month; allow user export for compliance

```
# Background cleanup (runs daily)
SELECT short_key, original_url FROM url_mappings
WHERE expires_at < NOW()
LIMIT 100000
FOR UPDATE SKIP LOCKED;

# Cursor-based pagination per shard
SELECT short_key FROM url_mappings
WHERE shard_id = ? AND short_key > ?
ORDER BY short_key LIMIT 10000;
```

Cursor-based cleanup query to avoid locking

Compliance

- **GDPR:** Right to delete: anonymize user_id, retain short_key for reference
- **Audit log:** Log all deletions (short_key, user_id, deleted_at, reason)
- **Retention:** Keep audit logs for 2 years minimum

© Study Guide — For personal use only

11 Security & Abuse Prevention

Protecting the system

Malicious URL Detection

- **Google Safe Browsing API:** Check against phishing, malware, unwanted software
- **Call on shorten:** Synchronously block unsafe URLs
- **Fallback:** Log to Kafka for manual review if API unavailable
- **Rate limit:** 5 requests/second per IP for Safe Browsing calls

Rate Limiting

- **Per-user:** Token bucket: 1000 shortened URLs per day per user
- **Per-IP:** 100 requests/minute (anonymous users)
- **Implementation:** Redis with INCR + EXPIRE; check on every request
- **Bypass:** Premium tier gets 10K/day; enterprise custom limits

```
# Token bucket rate limit (Redis)
key = f'rate_limit:{user_id}'
current = redis.get(key) or 0
if current >= limit:
return 429 # Too Many Requests
redis.incr(key)
redis.expire(key, 86400) # 1 day
```

Token bucket implementation in Redis

Private URLs & Access Control

- **Private flag:** Boolean column in url_mappings
- **Access check:** On read, if private, verify auth token matches owner
- **Shared access:** Maintain access_control table (url_id, user_id, expires_at)
- **Token generation:** JWT with user_id, url_id, expiry; sign with secret

DDoS Protection

- **CDN DDoS:** Cloudflare or AWS Shield; absorbs volumetric attacks
- **Application layer:** Rate limit per IP; detect patterns (same short_key)
- **Auto-scaling:** Scale Read Service based on request rate (threshold: 200K/sec)
- **Fallback:** Degrade analytics (skip Kafka log) if overloaded

SQL Injection Prevention

© Study Guide — For personal use only

- **Parameterized queries:** Always use prepared statements (Python: sqlalchemy ORM)
- **Input validation:** Reject short_key not matching ^[0-9a-zA-Z]{1,7}$
- **Original URL:** Validate URL format; reject if >2000 characters

© Study Guide — For personal use only

12 Key Generation Service (KGS)

Generating unique keys

Architecture

- **KGS instance:** Stateless service with in-memory key pool
- **Key pool DB:** Separate MySQL instance storing reserved keys (counter-based)
- **Pre-generation:** KGS generates 100K keys at startup
- **Local cache:** In-memory queue; refill when <10K remaining
- **Standby KGS:** Secondary instance for failover (heartbeat every 5 sec)

Key Pool Database Schema

```
CREATE TABLE key_pool (
pool_id INT PRIMARY KEY AUTO_INCREMENT,
start_counter BIGINT NOT NULL,
end_counter BIGINT NOT NULL,
status ENUM('AVAILABLE', 'RESERVED', 'EXHAUSTED'),
reserved_by VARCHAR(50), -- KGS instance ID
reserved_at TIMESTAMP,
INDEX idx_status (status)
);
```

Key pool table for tracking counter ranges

Key Reservation Flow

1. **KGS startup:** SELECT FROM key_pool WHERE status='AVAILABLE' LIMIT 1
2. **Reserve range:** UPDATE key_pool SET status='RESERVED', reserved_by=? WHERE pool_id=?
3. **Fetch keys:** Load start_counter to end_counter; convert to Base62
4. **Local cache:** Store in-memory queue; increment on each request
5. **Refill trigger:** When queue < 10K keys, SELECT next available pool

Atomicity & HA

- **Atomicity:** Each key issued exactly once (counter ranges reserved atomically)
- **Collision avoidance:** Non-overlapping ranges per KGS instance
- **Failover:** Standby KGS watches primary; takes over if heartbeat lost
- **No shared state:** Each KGS maintains local pool; no coordination needed

Edge Case: KGS Exhaustion

© Study Guide — For personal use only

- **Problem:** All 62^7 keys exhausted (unlikely in 100 years)
- **Solution 1:** Increase key length to 8 characters (62^8 = 218 trillion)
- **Solution 2:** Recycle expired keys (rescan and return to pool)
- **Monitoring:** Alert when pool utilization > 90%

© Study Guide — For personal use only

13 Scalability & Distribution

Growing to billions

Horizontal Scaling

- **Write Service:** Stateless; add instances behind load balancer
- **Read Service:** Stateless; scale independently from writes
- **KGS:** Multiple instances; each manages own key pool range
- **Database:** Shard by short_key hash (256–1024 shards)

Database Sharding

- **Shard key:** hash(short_key) % 256
- **Shard replication:** Primary + 2 replicas per shard (3-way)
- **Read replicas:** Serve analytics queries; eventual consistency acceptable
- **Write failover:** Automatic promotion of replica to primary (using Orchestrator)

Global Distribution

- **Primary region:** US East (for writes and strong consistency)
- **Read replicas:** US West, EU, APAC (cross-region, 1–2 sec lag)
- **CDN:** Global edge locations cache popular URLs
- **Kafka:** Multi-region cluster; replicate analytics across regions

Cache Strategy

- **Redis cluster:** 16 nodes, 100 GB RAM, sharded by short_key
- **Cache-aside pattern:** Read service checks Redis; miss = MySQL query + cache fill
- **TTL:** 24 hours for hot URLs; 1 hour for others
- **Invalidation:** Manual purge on custom URL update; periodic refresh

> **N CONSISTENT HASHING**
>
> Use **consistent hashing** for cache and database sharding. Allows adding/removing nodes with minimal key redistribution (only 1/N keys move). Example: Jump Hash or Ketama.

Monitoring & Alerts

- **Metrics:** Latency (p50, p99), error rate, throughput, cache hit %
- **Alerts:** >100ms latency, >1% error rate, <50% cache hit, KGS exhaustion
- **Tools:** Prometheus + Grafana for metrics; PagerDuty for on-call

© Study Guide — For personal use only

14 Failure Modes & Recovery

What can go wrong?

Failure	Impact	Detection	Mitigation
Redis down	Cache miss; 20x → MySQL load	Redis health check	Failover to replica; degrade gracefully
MySQL shard down	Reads/writes to that shard fail	Connection timeout	Auto-promote replica; reroute to other shards
KGS exhausted	Cannot create new URLs	KGS returns error	Scale KGS; increase key length; recycle old keys
CDN edge down	Slow redirects from that region	CDN provider alert	Reroute to nearest edge; fallback to origin
Kafka down	Analytics not logged; writes still work	Kafka broker dead	Async retry; local buffer + replay on recovery
Safe Browsing API timeout	Shorten request slow	HTTP timeout (>2sec)	Return success; log for async review
Network partition	Cross-region unavailable	Latency spike; connection errors	Local DC serves; eventual consistency on repair
Malicious URL spike	Safe Browsing API overload	Rate limit hit	Local cache of known malicious; batch requests

Disaster Recovery

- **RTO (Recovery Time Objective):** <30 minutes for any single service
- **RPO (Recovery Point Objective):** <1 minute for analytics (Kafka); <5 min for URLs (MySQL)
- **Backups:** Incremental MySQL backups every hour; S3 for 30-day retention
- **Failover:** Automatic for cache, KGS; manual for database (ensure consistency)

■ GRACEFUL DEGRADATION

In failures, prioritize **redirects** (core function). Degrade **analytics logging** (non-critical) and **custom aliases** (premium). Always return 200 for known URLs, 404 for unknown; never 5xx unless critical.

© Study Guide — For personal use only

15 Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Redirect type	301 Permanent	Browser cache reduces load, but cannot be changed later; 302 allows flexibility
Cache consistency	Eventual	Stale data briefly; faster reads; alternative: Strong (slower)
Analytics	Async (Kafka)	Delays 1–2 sec; does not block shorten; alternative: Sync (slows shorten by 50ms)
Database	MySQL sharded	Operational complexity; good consistency; alternative: NoSQL (easier but weaker consistency)
Key generation	Pre-generated (KGS)	Added service; no collisions; alternative: Base62 counter (simpler but distributed)
Expiry	Lazy	Stale data briefly; low overhead; alternative: Active (scan overhead)
CDN coverage	60% popular URLs	Reduced origin load; miss rate 40%; alternative: 100% (higher CDN cost)
Sharding	256 shards	Good parallelism; manageable operations; alternative: 64 (contention) or 1024+ (overhead)

© Study Guide — For personal use only

16 Interview Playbook

How to present this

This guide distills a 40-minute interview into a structured narrative. Use it to practice explaining system design clearly, defending trade-offs, and handling follow-up questions with confidence.

45-Minute Interview Outline

1. **Intro (2 min):** 'Let me clarify requirements: 100M URLs/day, 100:1 read:write, 5-year retention, global'
2. **Capacity (5 min):** '1.2K writes/sec, 120K reads/sec, 91 TB storage, cache 11.5 GB'
3. **High-level arch (3 min):** Show Diagram 1; explain CDN, LB, services, data tier
4. **Write path (8 min):** Show Diagram 2; walk through KGS, MySQL, Redis; explain atomicity
5. **Read path (7 min):** Show Diagram 3; explain cache hits, redirect type, analytics logging
6. **Deep dive (12 min):** Choose: KGS design (counter ranges, HA), sharding strategy, or analytics
7. **Scalability (5 min):** Horizontal scaling, global distribution, monitoring
8. **Failures (3 min):** Top 3 failure modes; mitigation for each

Key Talking Points

- **'1.2K writes/sec, 120K reads/sec':** Demonstrates capacity math
- **'62^7 = 3.5 trillion keys':** Shows why KGS works and doesn't exhaust
- **'91 TB storage':** Justifies database sharding strategy
- **'KGS pre-generates keys':** Prevents collisions; enables atomicity
- **'301 redirect + CDN cache':** Reduces load; handles 100:1 ratio
- **'Async Kafka for analytics':** Doesn't block critical path
- **'Eventual consistency acceptable':** Trade-off rationale

Common Follow-ups & Answers

Q: How do you prevent collisions?

A: KGS reserves keys from a pre-generated pool. Each key is issued exactly once; database UNIQUE constraint catches any duplicates (should never happen).

Q: What if MySQL goes down?

A: Reads go down for that shard. Automatic failover: promote replica to primary using Orchestrator. Writes are buffered locally; retry on recovery.

Q: How do you scale the cache?

© Study Guide — For personal use only

A: Redis cluster with 16 nodes, sharded by short_key. Consistent hashing allows adding nodes with minimal key redistribution.

Q: What's the cost of 301 vs 302?

A: 301 reduces requests by ~80% (browser cache). 302 requires every request to origin. At 120K reads/sec, 301 saves significant infrastructure; we accept inability to change destination.

Q: How do you handle malicious URLs?

A: Check with Google Safe Browsing API on shorten (blocking). Also rate-limit per user (1000/day) to prevent abuse.

Memorize These Numbers

- **1.2K writes/sec, 120K reads/sec** (from 100M/day, 100:1 ratio)
- **62^7 = 3.5 trillion keys** (why 7-char keys work)
- **91 TB storage over 5 years** (justifies sharding)
- **80% cache hit rate** (why Redis matters)
- **60% CDN hit rate** (popular URLs cached at edge)

Red Flags to Avoid

- **Don't mention:** 'Use NoSQL' without explaining trade-offs (consistency, complexity)
- **Don't ignore:** Failure modes and recovery (shows thinking about production)
- **Don't oversimplify:** Analytics as afterthought (20% of the challenge)
- **Don't forget:** Clarifying questions (show you understand requirements before designing)

Practice Tips

- **Draw diagrams:** Practice sketching architecture on whiteboard; explain as you draw
- **Time yourself:** 2 min clarify, 5 min capacity, 8 min deep-dive architecture, 8 min deep-dive component, 20 min for questions
- **Know your weak spot:** If sharding is weak, drill it; if analytics weak, drill it
- **Record yourself:** Practice explaining out loud; catch filler words ('um', 'like')
- **Anticipate pushback:** 'Why not use X?' 'What if Y fails?' Have 2-3 answers ready

© Study Guide — For personal use only